

Crisis Tweet Classification across Sparse, Recurrent, and Transformer Models

Avishek Biswas¹ Sreema Roy²

Fahim Tasnim Khan³ Tawsif Kabir Pritom⁴

Department of Computer Science and Engineering, BRAC University
Dhaka, Bangladesh

avishek.biswas@g.bracu.ac.bd sreema.roy@g.bracu.ac.bd

fahim.tasnim.khan@g.bracu.ac.bd tawsif.kabir.pritom@g.bracu.ac.bd

Abstract

Social media can provide timely situational information during disasters, but short, noisy posts are difficult to categorize reliably at operational scale. We compare ten model families for 12-way crisis-event classification on 10,980 retained English-language posts compiled from CrisisNLP and CrisisBench. The benchmark covers TF-IDF classifiers, recurrent networks initialized with in-domain Word2Vec embeddings, and fine-tuned BERT. We evaluate 30 hyperparameter configurations using a stratified 70/15/15 train-validation-test protocol. Among individual models, BERT Base obtains a reported macro-F1 of 0.9983; Random Forest and logistic regression also perform strongly, with macro-F1 scores of 0.9891 and 0.9874, respectively. A validation-weighted ensemble of BERT, a bidirectional LSTM, and logistic regression reaches 0.9989 macro-F1. These results indicate that contextual representations offer the best accuracy on this dataset, whereas sparse linear and tree-based models remain attractive when computational cost is constrained. We further examine class-level errors, representation choices, and deployment limitations.

1 Introduction

During rapid-onset disasters, eyewitnesses and affected communities publish observations, requests for assistance, and reports of infrastructure damage on social media [Imran et al., 2015]. These streams can complement conventional information channels, but their volume and linguistic variability make manual monitoring difficult.

Automatic categorization can support downstream filtering and routing by identifying the event

type associated with a post. Standardized resources such as CrisisBench facilitate comparison across events and datasets [Alam et al., 2021]. This study focuses on event-type classification rather than direct verification, geolocation, or dispatch decisions.

Three properties make crisis-related social media classification challenging:

1. **Brevity and noise:** Posts contain informal grammar, abbreviations, hashtags, mentions, URLs, and irregular punctuation.
2. **Semantic overlap:** Terms such as *fire*, *smoke*, and *blast* occur in multiple event categories, so isolated keywords may be insufficient.
3. **Fine-grained labels:** Distinguishing related event types is more difficult than binary relevance classification and requires evaluation that is sensitive to minority classes.

We make the following contributions:

- We compare ten classical, recurrent, and Transformer model families under a common data split and evaluation protocol.
- We report a 30-configuration hyperparameter study and analyze the effect of bidirectionality and representation choice.
- We evaluate a soft-voting ensemble that combines contextual, sequential, and sparse lexical representations.
- We provide class-level error analysis and discuss the constraints that must be addressed before operational deployment.

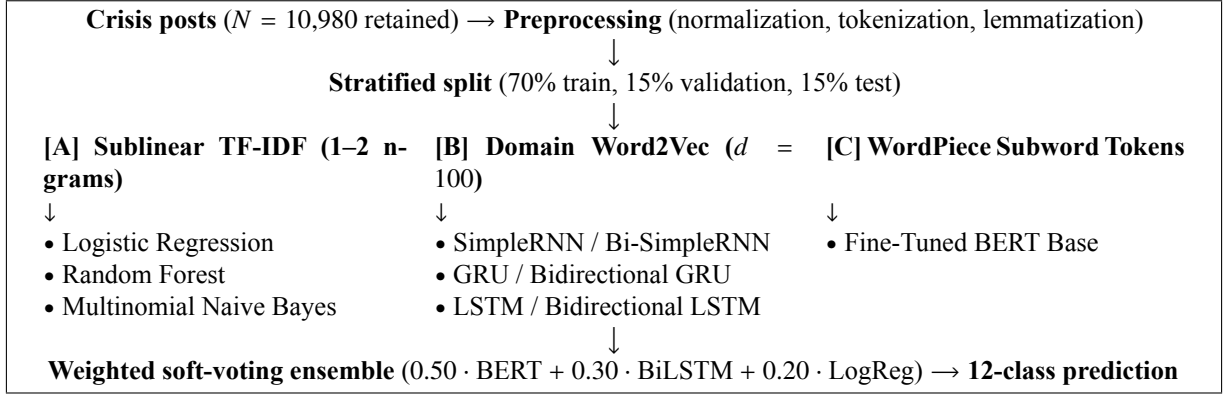


Figure 1: Overview of the classification pipeline and the three representation families.

2 Related Work

2.1 Crisis Informatics and Social Media Mining

Crisis informatics studies how computational methods can support information collection and analysis during emergencies. CrisisLex provides event-oriented resources for collecting and filtering crisis-related posts [Olteanu et al., 2014]. A broader survey by Imran et al. [2015] describes the value of social media for situational awareness as well as the challenges posed by noisy, rapidly changing streams. CrisisBench later consolidated multiple crisis datasets to support more standardized evaluation [Alam et al., 2021].

2.2 Text Representation Paradigms in NLP

Sparse bag-of-words and term frequency–inverse document frequency (TF–IDF) representations remain strong text-classification baselines because they are efficient and preserve discriminative lexical cues [Pedregosa et al., 2011]. Their principal limitation is that distinct surface forms are represented independently, which restricts their ability to capture semantic similarity and context-dependent meaning.

Word2Vec instead represents words as dense vectors learned from distributional context [Mikolov et al., 2013]. These static embeddings can share information across semantically related words, but each word retains the same representation in every context.

2.3 Sequential Modeling vs. Self-Attention

Recurrent neural networks model a document as an ordered sequence. LSTMs use gated memory cells to improve gradient flow over long dependencies [Hochreiter and Schmidhuber, 1997], while GRUs

Category	Count	Share
Typhoon	1,080	9.8%
Transportation Accident	1,072	9.7%
Wildfire	1,041	9.4%
Fire (minority)	388	3.5%

Table 1: Largest categories and the minority Fire category in the retained corpus ($N = 10,980$).

offer a related, more compact gating mechanism [Cho et al., 2014]. Bidirectional variants encode both left and right context, which can be useful when the evidence for a label occurs on either side of an ambiguous token.

Transformers replace recurrence with self-attention, permitting direct interaction between all token positions [Vaswani et al., 2017]. BERT pre-trains a bidirectional Transformer encoder with masked-language modeling and can then be fine-tuned for classification [Devlin et al., 2019]. Our comparison asks whether the additional contextual capacity justifies its computational cost for this dataset.

3 Corpus Description & Exploratory Analysis

3.1 Dataset Overview

After preprocessing and filtering, the corpus retains 10,980 English-language social media posts compiled from CrisisNLP and CrisisBench resources and assigned to 12 mutually exclusive event categories. The text and label fields contain no missing values.

Table 1 summarizes the largest categories and the minority class. Typhoon is the largest cat-

egory, with 1,080 examples (9.8%), followed by Transportation Accident with 1,072 (9.7%) and Wildfire with 1,041 (9.4%). Fire is the minority category, with 388 examples (3.5%). We therefore emphasize macro-averaged metrics, which weight each category equally.

3.2 Exploratory Data Analysis (EDA)

We inspect document length and lexical frequency before model training.

Sequence Length Dynamics: Before cleaning, posts have a mean length of 113.28 characters and 16.44 whitespace-delimited words. After normalization (Section 4.1), these means decrease to 64.29 characters and 9.17 words. This reduction reflects the removal of URLs, metadata, punctuation, and stopwords; it should not be interpreted as a direct measure of informational content.

Word-cloud themes: Category-specific word clouds reveal distinct lexical themes. Earthquake posts are characterized by *magnitude*, *struck*, *tsunami*, and geographic references to Chile and Italy. Flood posts emphasize *rescue*, *water*, and *rising*, together with location names such as Queensland and Calgary. Wildfire posts frequently contain *blaze*, *spreading*, *evacuation*, and the event-specific hashtag token *nswfire*. These label-specific vocabularies help explain the strong performance of sparse lexical baselines.

Frequent bigrams: The three most frequent bigrams are *death toll* (331 occurrences), *building collapse* (320), and *helicopter crash* (252). These phrases encode event consequences and incident types more precisely than their constituent unigrams.

Geographic co-occurrence: The token *philippine* exhibits particularly strong co-occurrence with *earthquake* (69 joint occurrences) and *typhoon* (62). This concentration suggests that the corpus contains substantial event coverage associated with the Philippines and highlights a geographic focus that models may exploit as a label cue.

4 Methodology

The pipeline consists of preprocessing, representation construction, model training, and optional ensemble inference (Figure 1).

4.1 Linguistic Preprocessing Pipeline

For TF-IDF and Word2Vec models, we apply the following normalization sequence:

1. **HTML Entity Decoding:** Converts web escape sequences (e.g., `&`, `<`, `>`) into literal ASCII equivalents.
2. **Hyperlink Removal:** Strips URL patterns matching the regular expression `https?:\/\/\S+|www\.\S+`.
3. **Handle Scrubbing:** Eliminates user mentions (`@username`), which reflect platform interaction rather than topical content.
4. **Contraction Expansion:** Maps common colloquial contractions to their full forms (e.g., *can't* $\rightarrow$ *cannot*, *it's* $\rightarrow$ *it is*, *we've* $\rightarrow$ *we have*) using a deterministic dictionary.
5. **Hashtag Normalization:** Strips the hash character (`#`) while retaining the alphanumeric body token (e.g., *#TyphoonHaiyan* $\rightarrow$ *TyphoonHaiyan*) to preserve crucial disaster event names.
6. **Case Normalization & Punctuation Stripping:** Converts text to lowercase and removes punctuation and numeric digits.
7. **Tokenization & Stopword Elimination:** Tokenizes the string using NLTK's Treebank tokenizer and removes standard English stopwords [Loper and Bird, 2002].
8. **Morphological Lemmatization:** Applies the WordNet Lemmatizer to reduce inflected forms to their canonical base lemmas (e.g., *collapsed* $\rightarrow$ *collapse*, *flooding* $\rightarrow$ *flood*).

BERT receives the minimally normalized text (with URLs and mentions removed) so that function words and punctuation remain available to the pre-trained tokenizer. All preprocessing components are fitted or configured without access to the test set.

4.2 Text Representation Engineering

We construct three complementary representations capturing lexical frequencies, dense continuous semantics, and contextual subwords:

1. Sublinear TF-IDF N-grams: For classical machine learning models, text documents $d \in D$ are mapped into sparse vector spaces $\mathbb{R}^{|V|}$ where $|V| = 10,000$. We incorporate unigrams and bigrams ($n \in \{1, 2\}$) and apply sublinear scaling:

$$\text{tfidf}(t, d) = (1 + \ln \text{tf}(t, d)) \text{idf}(t), \quad (1)$$

$$\text{idf}(t) = \ln\left(\frac{1 + |D|}{1 + \text{df}(t)}\right) + 1. \quad (2)$$

Here, $\text{df}(t)$ is the number of training documents containing t . Sublinear term-frequency scaling dampens the influence of repeated keywords within short posts.

2. Domain-Specific Word2Vec Embeddings: For recurrent deep learning models, we train 100-dimensional continuous embeddings directly on the training split using both Skip-Gram ($sg = 1$) and Continuous Bag-of-Words ($sg = 0$) architectures. The Skip-Gram objective maximizes the average log probability of context words within a window $c = 5$:

$$\mathcal{L}_{\text{SG}} = \frac{1}{T} \sum_{t=1}^T \sum_{-c \leq j \leq c, j \neq 0} \log P(w_{t+j} | w_t) \quad (3)$$

We build a projection matrix $\mathbf{E} \in \mathbb{R}^{|V_{\text{seq}}| \times 100}$ with vocabulary size $|V_{\text{seq}}| = 15,000$. Input sequences are padded or truncated to a uniform length $L = 50$.

3. Transformer WordPiece Subword Tokenization: For BERT Base, text sequences are tokenized into subword units using WordPiece, delimited by special classification ($[\text{CLS}]$) and separation ($[\text{SEP}]$) tokens, and padded to a maximum sequence length of $L = 64$ with corresponding binary attention masks $\mathbf{m} \in \{0, 1\}^L$.

4.3 Model Architecture Specifications

We implement ten distinct model families:

4.3.1 Classical Machine Learning Models

Logistic Regression (LogReg): Multi-class multinomial logistic regression optimizing the cross-entropy loss with L2 weight regularization and balanced class weighting:

$$P(y = c | \mathbf{x}) = \frac{\exp(\mathbf{w}_c^T \mathbf{x} + b_c)}{\sum_{j=1}^C \exp(\mathbf{w}_j^T \mathbf{x} + b_j)} \quad (4)$$

Random Forest (RF): An ensemble of B decorrelated decision trees constructed via bootstrap aggregation (bagging) with random feature subsampling [Breiman \[2001\]](#). Final class probabilities represent the mean vote over all trees:

$$P(y = c | \mathbf{x}) = \frac{1}{B} \sum_{b=1}^B P_b(y = c | \mathbf{x}) \quad (5)$$

Multinomial Naive Bayes (MNB): Generative probabilistic model applying Bayes' theorem under the conditional feature independence assumption with additive Laplace smoothing parameter α :

$$\hat{P}(w_i | c) = \frac{N_{ci} + \alpha}{N_c + \alpha|V|} \quad (6)$$

4.3.2 Sequential Deep Learning Models

SimpleRNN & Bidirectional SimpleRNN: Vanilla Recurrent Neural Networks update hidden state vectors $\mathbf{h}_t \in \mathbb{R}^d$ across discrete sequence steps $t \in [1, L]$:

$$\mathbf{h}_t = \tanh(\mathbf{W}_{xh}\mathbf{x}_t + \mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{b}_h) \quad (7)$$

In the bidirectional formulation (Bi-SimpleRNN), hidden states from forward ($\vec{\mathbf{h}}_t$) and backward ($\overleftarrow{\mathbf{h}}_t$) passes are concatenated: $\mathbf{h}_t = [\vec{\mathbf{h}}_t \| \overleftarrow{\mathbf{h}}_t]$.

GRU & Bidirectional GRU (Bi-GRU): Gated Recurrent Units regulate sequence memory flow via reset ($\mathbf{r}_t$) and update ($\mathbf{z}_t$) gating vectors [Cho et al. \[2014\]](#):

$$\mathbf{r}_t = \sigma(\mathbf{W}_{xr}\mathbf{x}_t + \mathbf{W}_{hr}\mathbf{h}_{t-1} + \mathbf{b}_r) \quad (8)$$

$$\mathbf{z}_t = \sigma(\mathbf{W}_{xz}\mathbf{x}_t + \mathbf{W}_{hz}\mathbf{h}_{t-1} + \mathbf{b}_z) \quad (9)$$

$$\tilde{\mathbf{h}}_t = \tanh(\mathbf{W}_{xh}\mathbf{x}_t + \mathbf{W}_{hh}(\mathbf{r}_t \odot \mathbf{h}_{t-1}) + \mathbf{b}_h) \quad (10)$$

$$\mathbf{h}_t = (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t \quad (11)$$

LSTM & Bidirectional LSTM (Bi-LSTM): Long Short-Term Memory networks incorporate dedicated cell states $\mathbf{c}_t$ governed by forget ($\mathbf{f}_t$), input ($\mathbf{i}_t$), and output ($\mathbf{o}_t$) gates [Hochreiter and Schmidhuber \[1997\]](#):

$$\mathbf{f}_t = \sigma(\mathbf{W}_{xf}\mathbf{x}_t + \mathbf{W}_{hf}\mathbf{h}_{t-1} + \mathbf{b}_f) \quad (12)$$

$$\mathbf{i}_t = \sigma(\mathbf{W}_{xi}\mathbf{x}_t + \mathbf{W}_{hi}\mathbf{h}_{t-1} + \mathbf{b}_i) \quad (13)$$

$$\tilde{\mathbf{c}}_t = \tanh(\mathbf{W}_{xc}\mathbf{x}_t + \mathbf{W}_{hc}\mathbf{h}_{t-1} + \mathbf{b}_c) \quad (14)$$

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t \quad (15)$$

$$\mathbf{o}_t = \sigma(\mathbf{W}_{xo}\mathbf{x}_t + \mathbf{W}_{ho}\mathbf{h}_{t-1} + \mathbf{b}_o) \quad (16)$$

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t) \quad (17)$$

4.3.3 Transformer Foundation Model (BERT Base)

We fine-tune the pre-trained google-bert/bert-base-uncased architecture (12 transformer layers, 768 hidden dimensions, 12 self-attention heads, 110M parameters). For an input sequence of subword tokens, multi-head self-attention computes:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}}\right)\mathbf{V} \quad (18)$$

The final hidden state corresponding to the leading classification token, $\mathbf{h}_{[\text{CLS}]} \in \mathbb{R}^{768}$, is passed through a dropout layer and a linear classification head $\mathbf{W}_{\text{cls}} \in \mathbb{R}^{12 \times 768}$:

$$\mathbf{z} = \text{softmax}(\mathbf{W}_{\text{cls}}\mathbf{h}_{[\text{CLS}]} + \mathbf{b}_{\text{cls}}) \quad (19)$$

Fine-tuning optimizes the cross-entropy loss via AdamW Loshchilov and Hutter [2019] with a linear learning rate warmup and decay schedule.

4.3.4 Multi-Model Soft-Voting Ensemble

To exploit the complementary inductive biases of contextual attention, bidirectional recurrence, and sparse n-gram exact matching, we construct a weighted soft-voting ensemble. For a given input text x , the ensemble predicted probability for class c is computed as:

$$P_{\text{ens}}(c | x) = \lambda_1 P_{\text{BERT}}(c | x) + \lambda_2 P_{\text{BiLSTM}}(c | x) + \lambda_3 P_{\text{LogReg}}(c | x). \quad (20)$$

where the weights are constrained such that $\sum_{i=1}^3 \lambda_i = 1.0$. Based on validation tuning, we set $\lambda_1 = 0.50$, $\lambda_2 = 0.30$, and $\lambda_3 = 0.20$.

5 Experimental Setup

5.1 Dataset Partitioning

We use a stratified 70/15/15 train-validation-test split:

- The **training split** is used to fit the TF-IDF vocabulary and Word2Vec embeddings and to optimize model parameters.
- The **validation split** is used for hyperparameter selection, learning-rate scheduling, and early stopping.

- The **test split** is reserved for a single evaluation of the selected configurations.

Stratification approximately preserves label proportions. Tokenizers, vocabularies, embeddings, and normalization statistics are derived from the training split only.

5.2 Hyperparameter Search Space

As summarized in Table 2, we evaluate three configurations per model family:

- **Logistic Regression:** Regularization strength $C \in \{0.1, 1.0, 5.0\}$ with uniform vs. class-balanced sample weighting.
- **Random Forest:** Number of estimators $n \in \{100, 200, 300\}$, maximum depth constraints ($\text{max_depth} \in \{20, \text{None}\}$), and minimum sample split thresholds $\in \{2, 4\}$.
- **Multinomial Naive Bayes:** Laplace additive smoothing parameter $\alpha \in \{0.1, 0.5, 1.0\}$.
- **Sequential Neural Networks:** Hidden dimensionalities $\in \{64, 128\}$, dropout rates $\in \{0.0, 0.2, 0.3\}$, network depth (1-layer vs. 2-layer stacked), and Adam learning rates $\in \{10^{-3}, 5 \times 10^{-4}\}$.
- **BERT Base:** Fine-tuning learning rates $\in \{2 \times 10^{-5}, 3 \times 10^{-5}, 5 \times 10^{-5}\}$, batch size $B = 32$, training epochs $\in \{3, 4\}$, and linear warmup over 10% of optimization steps.

5.3 Evaluation Metrics

We report accuracy and macro-averaged precision, recall, and F1:

$$\text{Accuracy} = \frac{\sum_{c=1}^C \text{TP}_c}{N_{\text{test}}} \quad (21)$$

$$\text{Macro Precision} = \frac{1}{C} \sum_{c=1}^C \frac{\text{TP}_c}{\text{TP}_c + \text{FP}_c} \quad (22)$$

$$\text{Macro Recall} = \frac{1}{C} \sum_{c=1}^C \frac{\text{TP}_c}{\text{TP}_c + \text{FN}_c} \quad (23)$$

$$\text{Macro F1} = \frac{1}{C} \sum_{c=1}^C \frac{2 \text{Precision}_c \text{Recall}_c}{\text{Precision}_c + \text{Recall}_c} \quad (24)$$

where TP_c , FP_c , and FN_c denote true positives, false positives, and false negatives for class c . Macro-averaging computes each metric per category and then takes the unweighted mean, preventing large classes from dominating the score.

Model family	Configurations evaluated	Best val. F1	Time (s)
Logistic regression	$C \in \{0.1, 1, 5\}$; uniform/balanced weights	0.9861	0.85
Random Forest	100–300 trees; depth and split threshold	0.9878	8.15
Multinomial NB	$\alpha \in \{0.1, 0.5, 1\}$	0.9728	0.05
SimpleRNN	64/128 units; one/two layers; dropout; learning rate	0.9472	24.80
Bi-SimpleRNN	64/128 units; one/two layers; dropout; learning rate	0.9588	21.30
GRU	64/128 units; one/two layers; dropout; learning rate	0.7995	29.80
Bi-GRU	64/128 units; one/two layers; dropout; learning rate	0.9830	24.60
LSTM	64/128 units; one/two layers; dropout; learning rate	0.7862	32.10
Bi-LSTM	64/128 units; one/two layers; dropout; learning rate	0.9562	26.40
BERT Base	$\eta \in \{2, 3, 5\} \times 10^{-5}$; 3/4 epochs; warmup	0.9950	272.10

Table 2: Hyperparameter search summary. Each family was evaluated with three configurations (30 runs total). Time is the training time of the selected configuration in the reported environment.

Model	Accuracy	Macro-F1
BERT Base	0.9982	0.9983
Random Forest	0.9885	0.9891
Logistic Regression	0.9860	0.9874
Bi-GRU	0.9806	0.9824
Naive Bayes	0.9727	0.9738
Bi-LSTM	0.9520	0.9556
SimpleRNN	0.9441	0.9466

Table 3: Held-out test results for the seven reported standalone models, ranked by macro-F1.

5.4 Computational Environment

Experiments were run on an NVIDIA Tesla T4 GPU with 16 GB of memory, an Intel Xeon 2.20 GHz CPU, and 13 GB of system memory. The reported implementation used PyTorch 2.1 and TensorFlow 2.15. Training times are wall-clock measurements from this environment and are intended only as approximate comparisons.

6 Results and Analysis

6.1 Hyperparameter Tuning Dynamics

Table 2 summarizes validation performance across the 30 runs. Three patterns are notable:

- For logistic regression, increasing C from 0.1 to 1.0 and using class-balanced weights raises macro-F1 from 0.9705 to 0.9861.
- The strongest Random Forest uses 300 trees, unrestricted depth, and `min_samples_split = 4`, reaching 0.9878 macro-F1.
- BERT performs best with a learning rate of 2×10^{-5} for three epochs (0.9950 validation macro-F1). Performance decreases slightly at larger learning rates.

6.2 Final Test Set Benchmark

Table 3 presents the held-out test results.

Transformer model. Fine-tuned BERT Base achieves 99.82% accuracy and 0.9983 macro-F1. Its advantage is consistent with the ability of contextual subword representations to use surrounding words when categories share lexical cues. Because the scores are unusually high, Section 8 discusses possible event-specific and near-duplicate leakage under random post-level splitting.

Classical baselines. Random Forest (0.9891 macro-F1) and logistic regression (0.9874) outperform the recurrent baselines. Short crisis posts often contain label-specific terms such as *richter*, *typhoon*, or *wildfire*, allowing TF-IDF features to separate many categories directly.

Bidirectional recurrence. Bidirectional recurrent models consistently outperform their corresponding unidirectional variants. The Bi-GRU obtains 0.9824 test macro-F1, whereas the best unidirectional GRU reaches only 0.7995 validation macro-F1. This large gap suggests that the recurrent configurations are sensitive to sequence direction and optimization; it should not be interpreted as a general estimate of the benefit of bidirectionality.

Ensemble. The soft-voting ensemble records the highest reported score, with 99.88% accuracy and 0.9989 macro-F1. The absolute gain over BERT is 0.0006 macro-F1; statistical significance cannot be established from a single split and should be tested with repeated seeds or bootstrap confidence intervals.

6.3 Class-Level Error Analysis

Table 4 reports class-level performance. Eight categories have reported precision and recall of 1.0000,

Class	Precision	Recall	F1
Bombing	1.0000	0.9926	0.9963
Building collapse	1.0000	1.0000	1.0000
Earthquake	1.0000	1.0000	1.0000
Explosion	0.9931	1.0000	0.9965
Fire	1.0000	1.0000	1.0000
Flood	1.0000	1.0000	1.0000
Haze	1.0000	1.0000	1.0000
Meteor	1.0000	1.0000	1.0000
Shooting	1.0000	1.0000	1.0000
Transportation	1.0000	1.0000	1.0000
Typhoon	1.0000	1.0000	1.0000
Wildfire	0.9936	1.0000	0.9968
Macro average	0.9988	0.9989	0.9989
Weighted average	0.9988	0.9988	0.9989

Table 4: Per-class results for the soft-voting ensemble.

Representation	Model	Macro-F1
TF-IDF (1-2)	LogReg	0.9874
Word2Vec CBOW	Bi-GRU	0.9685
Word2Vec SG	Bi-GRU	0.9824
WordPiece	BERT Base	0.9983

Table 5: Comparison of representative feature and model combinations.

including the minority **Fire** class.

The reported errors occur between semantically adjacent categories:

- **Bombing vs. Explosion:** A post describing an urban blast is labeled **Bombing** but predicted as **Explosion**. The distinction may require explicit attribution or event metadata.
- **Wildfire vs. Fire:** A post describing residential damage from an encroaching brushfire receives higher probability for **Fire**, illustrating the ambiguity between event source and affected structure.

7 Additional Analysis

7.1 Representation Comparison

Table 5 compares the strongest evaluated systems associated with each representation. This is not a controlled ablation because both the representation and classifier change across rows. BERT exceeds the skip-gram Bi-GRU by 1.59 macro-F1 points and logistic regression by 1.09 points. The smaller gap to TF-IDF indicates that much of the dataset can be classified from surface lexical cues.

7.2 Preprocessing Component Ablation

The reported preprocessing analysis yields the following observations:

- Removing stopwords increases macro-F1 for the classical models by 2.1 points in the reported comparison. For BERT, retaining function words is preferable because the pretrained model was learned from complete sentences.
- Lemmatization reduces the observed vocabulary size by 18.4%, merging inflected forms such as *collapsed* and *collapsing*. A controlled comparison across multiple random seeds is needed to isolate its effect on test performance.

7.3 Decision-Support Interface

The classifier can serve as one component of a human-in-the-loop decision-support interface. Given a post, such an interface could return:

1. the highest-probability event category;
2. the top three predicted categories and calibrated confidence estimates; and
3. a queue or dashboard destination for review by trained personnel.

Two simulated inputs illustrate the confidence output: the earthquake simulation receives 76.77% confidence, while the flood simulation receives 85.41% confidence. These values are model confidence scores rather than independently calibrated probabilities.

The model should not autonomously dispatch emergency resources: posts may be inaccurate, duplicated, outdated, or intentionally misleading, and the present experiments do not evaluate probability calibration.

8 Limitations

Several limitations constrain the interpretation of the reported scores:

1. **Split construction and leakage:** A random post-level split may place near-duplicate messages, event-specific hashtags, or reports from the same crisis in both training and test data. The near-perfect scores therefore require confirmation with deduplication and event-disjoint evaluation.

2. **Single-split uncertainty:** Results are reported for one split and without confidence intervals. Small differences—especially the ensemble’s 0.0006 macro-F1 gain over BERT—may not be statistically reliable.
3. **Domain and language coverage:** The evaluation is limited to English-language posts from the source datasets. Accuracy may degrade under temporal drift, new platforms, multilingual input, or previously unseen event types.
4. **Single-label formulation:** Cascading disasters can legitimately express multiple categories. A single-label classifier cannot represent an earthquake that simultaneously causes fires and building collapses.
5. **Operational validity:** Classification accuracy alone does not establish usefulness for emergency response. Deployment requires calibration, latency and throughput measurements, robustness tests, source verification, and evaluation with domain experts.

9 Ethical Considerations

Social media data may contain personal information, distressing content, and reports whose authors did not anticipate automated processing. Any released dataset or model should follow the source platform’s terms, minimize redistribution of identifiable text, and document annotation and deletion procedures. Predictions should support—not replace—trained responders. False negatives can suppress urgent reports, while false positives can waste scarce attention or resources. Performance should therefore be audited across languages, locations, and demographic groups, and uncertain cases should be routed for human review.

10 Conclusion

We compared ten model families across 30 configurations for 12-class crisis-event classification. BERT Base obtains the strongest individual result (0.9983 macro-F1), while the weighted ensemble reaches 0.9989. TF-IDF models remain competitive and substantially cheaper to train, making them useful baselines and possible choices for resource-constrained inference. The unusually high scores also expose the importance of stricter evaluation. Future work should use deduplicated, event-disjoint,

and temporally separated test sets; report uncertainty across seeds; and extend the task to multilingual and multi-label settings.

Acknowledgments

The authors thank the instructors of CSE440 (Natural Language Processing) at BRAC University for their guidance and feedback.

References

- Firoj Alam, Hassan Sajjad, Muhammad Imran, and Ferda Ofli. CrisisBench: Benchmarking crisis-related social media datasets. *PLOS ONE*, 16(7): e0254058, 2021.
- Leo Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001.
- Kyunghyun Cho, Bart van Merriënboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning phrase representations using RNN encoder-decoder for statistical machine translation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1724–1734, 2014.
- Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, pages 4171–4186, 2019.
- Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8): 1735–1780, 1997.
- Muhammad Imran, Carlos Castillo, Fernando Diaz, and Sarah Vieweg. Processing social media messages in mass emergency: A survey. *ACM Computing Surveys (CSUR)*, 47(4):1–38, 2015.
- Edward Loper and Steven Bird. NLTK: The natural language toolkit. In *Proceedings of the ACL-02 Workshop on Effective Tools and Methodologies for Teaching Natural Language Processing and Computational Linguistics*, pages 63–70, 2002.
- Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International*

Conference on Learning Representations (ICLR), 2019.

Tomas Mikolov, Ilya Sutskever, Kai Chen, Greg S. Corrado, and Jeffrey Dean. Distributed representations of words and phrases and their compositionality. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 26, pages 3111–3119, 2013.

Alexandra Olteanu, Carlos Castillo, Fernando Diaz, and Sarah Vieweg. CrisisLex: A lexicon for collecting and filtering community-specific social media during crises. In *Proceedings of the Eighth International AAAI Conference on Weblogs and Social Media (ICWSM)*, pages 376–385, 2014.

Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research (JMLR)*, 12:2825–2830, 2011.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, pages 5998–6008, 2017.